



Observability

Stefan Schmöller

stefan.schmoeller@qaware.de

What exactly is observability?

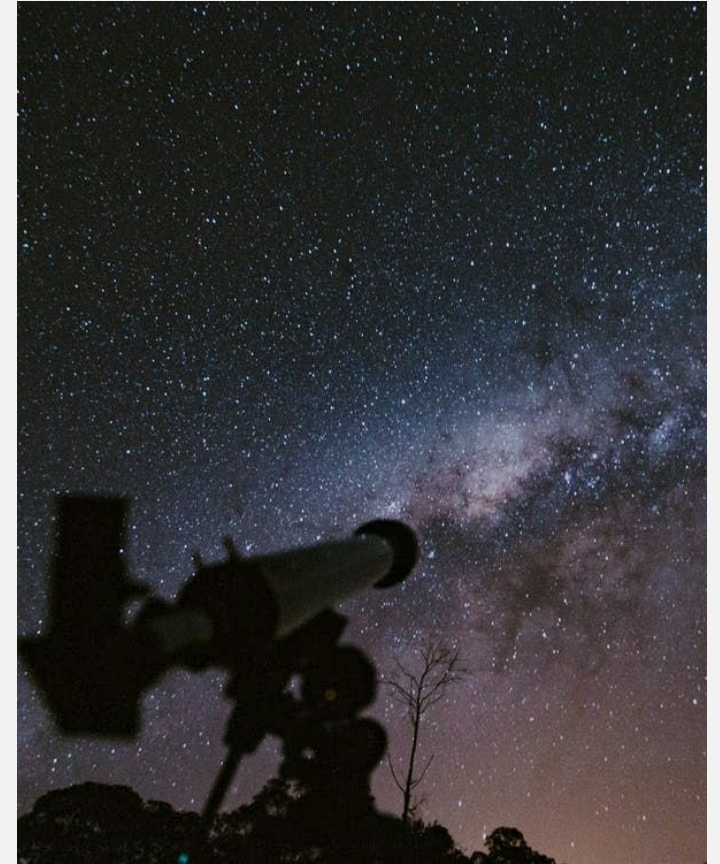


How well do I know the internal state of my system when I look at its outputs? (see [Kal])

What does such a modern system output?

- A website?
- A REST API?
- E-mails to the admin?
- A sparsely maintained log file?

What if the system consists of many microservices?



What exactly is observability?



“Observability” today typically refers to three topics, the so-called three pillars of observability:

- Logs
- Metrics
- Traces

All together, this enables insight into distributed microservices.

Sometimes those are also referred to as the four pillars, extended with

- Profiling



The goal of observability: Diagnosability. What do I understand by that?



QA|WARE

*“A **structured approach** to equipping an application with **sensors** in advance so that I can **quickly identify the error** in the event of a **fault** and have the information necessary to repair it.”*

A system can be easily diagnosed if you can easily recognize and correct healthy and unhealthy conditions.



A diagnosable system has a



- short Mean Time to Detect (MTDD)
- short Mean Time to Repair (MTTR)

and thus to a short period of time in which errors remain undetected and even exist.

Diagnosability: A structured approach is necessary to avoid influencing the system too much.



1. Get an overview:

- a. What are the central components, e.g. login, etc.?
- b. What are the supporting components, e.g. batch and loader jobs, etc.?

2. Identify error limits:

- a. Internal error limits: layers/use cases
- b. External error limits: incoming and outgoing calls

3. Define error classes:

- a. Severity levels: operation still possible, no impact on customer, etc.
- b. Impact: certain functions are not available to customers, etc.

4. Determine the runtime data that is necessary for detection:

- a. Uniformity: data has the same meaning; uniform data formats are defined, etc.
- b. Clearly defined: it is clear what the metric means, e.g. CPU load

5. Define actions:

- a. Alerting: who is notified and when
- b. Create playbooks for errors: How do I get the data, etc.



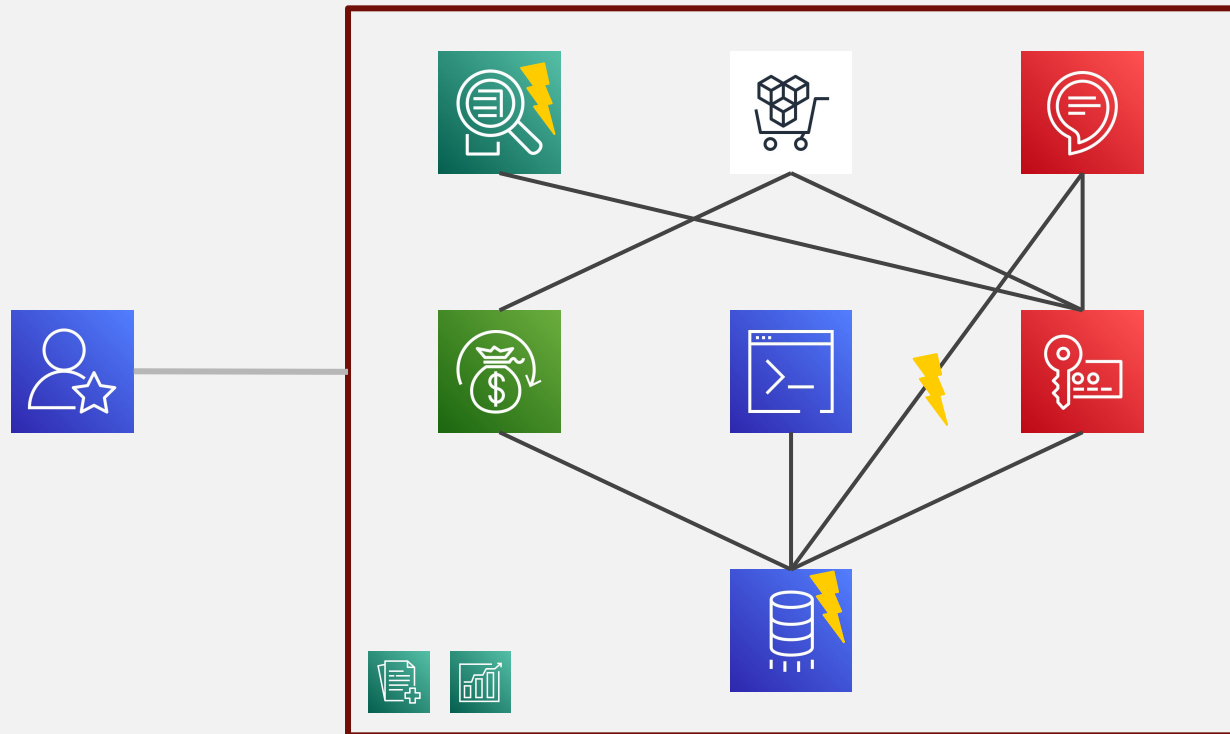
QA|WARE

Why (Cloud) Observability?

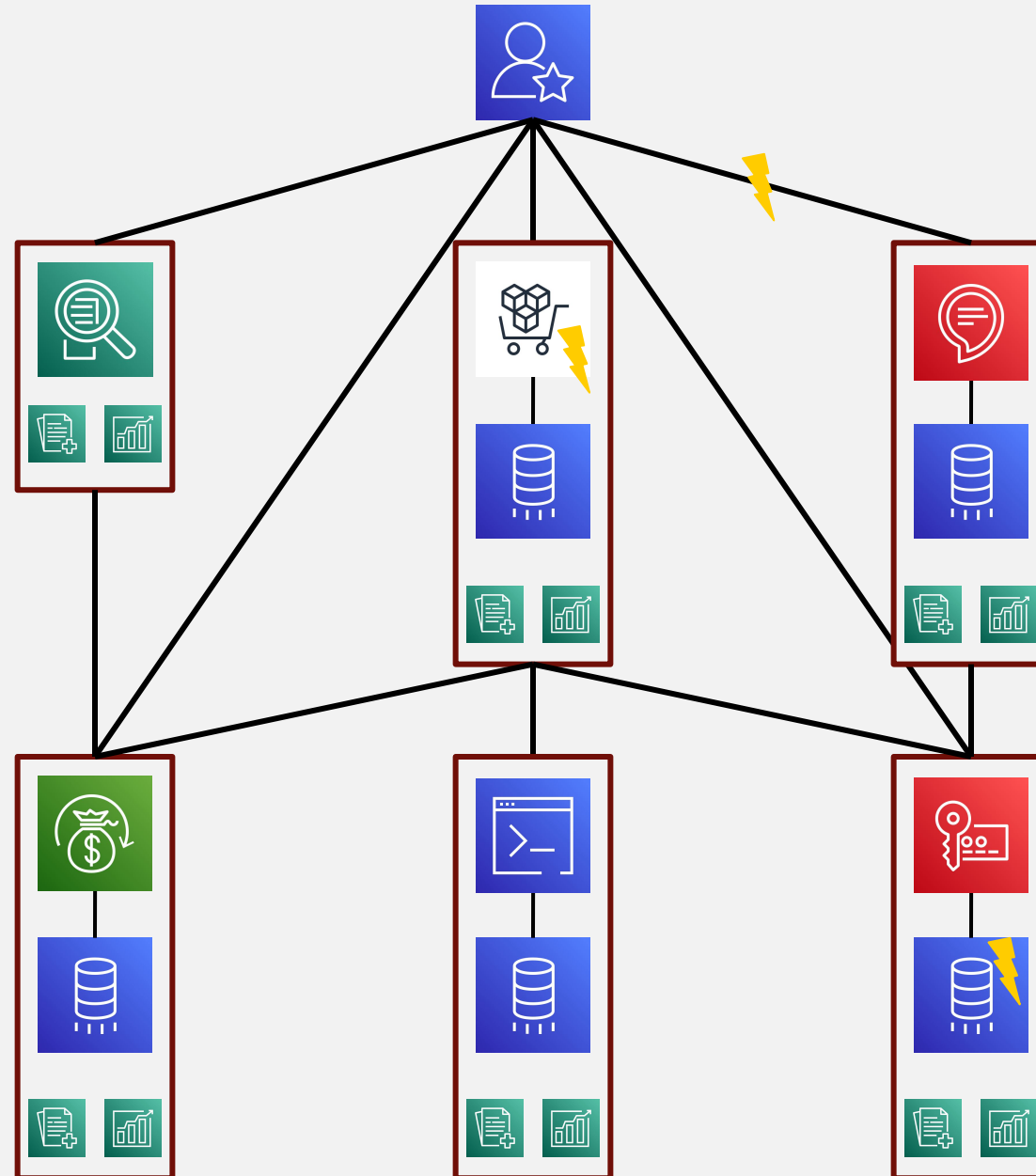
Back then The monolith



QA|WARE



Today Microservices



The (Open Source) Stack



QA|WARE

Today we use e.g. the LGTM stack:

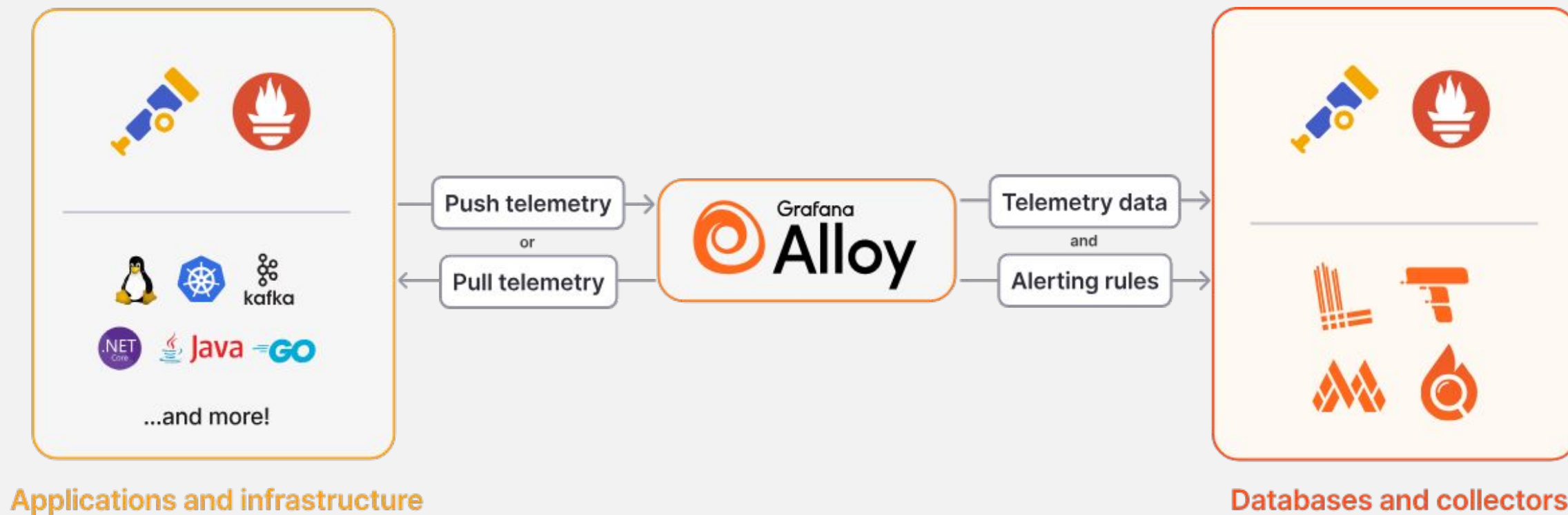
- Loki (Logs)
- Grafana (Visualization)
- Tempo (Traces)
- Mimir (Metrics)
- Pyroscope (Profiles)

<https://grafana.com/oss/>

The (Open Source) Stack



QA|WARE



The (Open Source) Stack - exercise

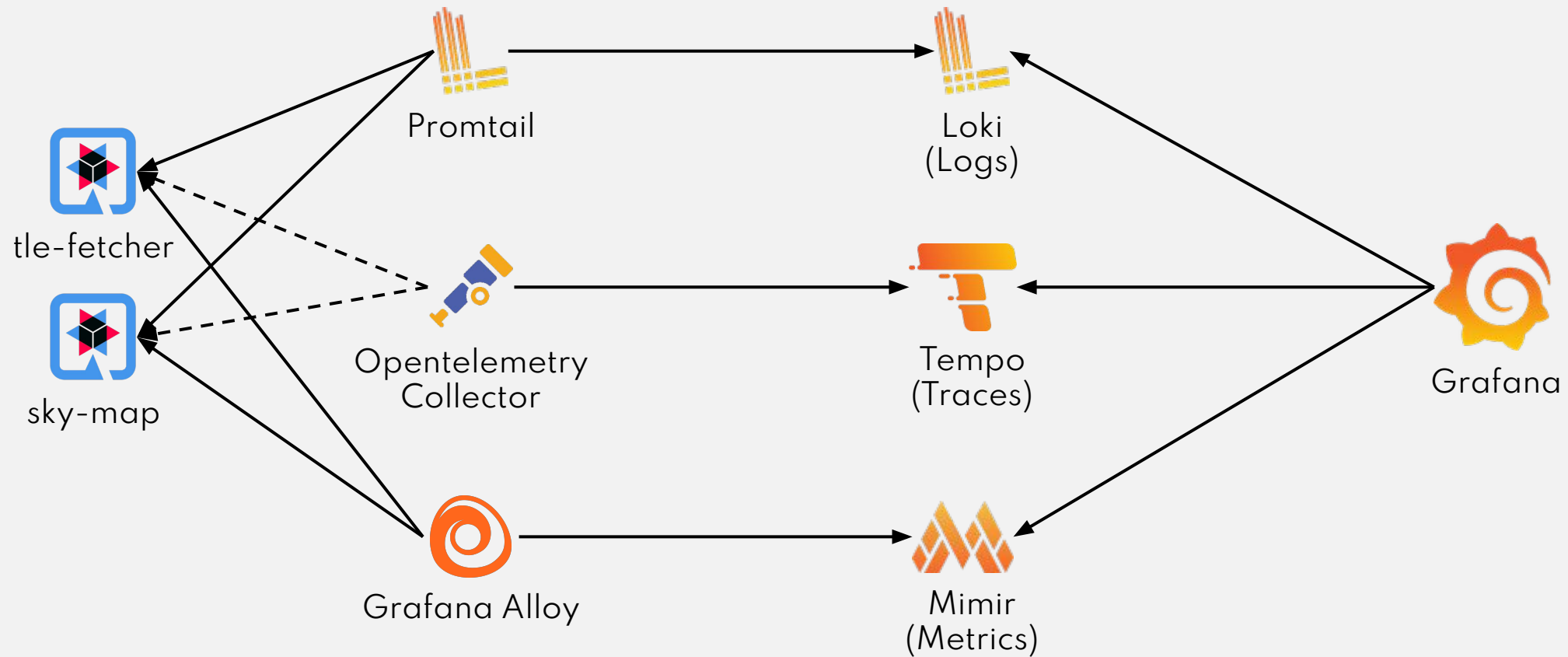


Applications

Data collection

Storage

Visualization





QA|WARE

Logs



Grafana loki

Logs



QA|WARE

- **Logs** are an important part of any IT system.
- In the cloud, we need to look at logs from many different services at the same time.
- Every system has logs, but they are always in a different **format**.
- Example: **Quarkus** webservice:
 - `2023-01-10 20:56:42,122 DEBUG [io.qua.mic.run.bin.ver.VertxHttpServerMetrics]
(vert.x-eventloop-thread-11) requestRouted null HttpRequestMetric [initialPath=/q/metrics,
currentRoutePath=null, templatePath=null, request=io.vertx.core.http.impl.Http1xServerRequest@512b1fd6]`
- Example: **Nginx** webserver:
 - `- 2a02:c207:3005:5132::1 - - [10/Jan/2023:00:00:13 +0100] 0.000 repo.saturn.codefoundry.de "POST
/api/v4/jobs/request HTTP/1.1" 957 204 0 "-" "gitlab-runner 15.6.1 (15-6-stable; go1.18.8; linux/amd64)"`

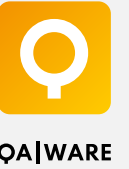
Logging. Please be structured. Please be well-considered. /1



QAWARE

- Define a log format and ensure that all services use the same format.
- Define a diagnostic context = information that helps in the event of an error, e.g.
 - Traceld
 - UserId
 - SessionId
- Use structured logging (e.g. JSON)
 - Simplifies handling in the analysis
- Use asynchronous logging (provided that the recipients support it) to prevent blockages
 - If all of this is sent over TCP etc. and then filtered and sorted later, the order doesn't matter 😎

Logging. Please be structured. Please be well-considered. /2



- Don't log too much, but log every
 - exception,
 - error,
 - meaningful piece of information ...
 - ... but not multiple times.
- Use log levels. To do this, define which category should have which level, e.g.:
 - WARN: Errors that affect a single request but not the stability of the service.
 - ERROR: Errors that affect the stability of the service.

Loki



QAWARE

Loki is the log aggregator in the Grafana ecosystem.

Passive log storage – logs have to be obtained differently.

Grafana **Alloy** is responsible for collecting and forwarding logs.

Alloy configuration



QAWARE

```
local.file_match "local_files" {
  path_targets = [{"__path__" = "/var/log/*.log"}]
  sync_period = "5s"
}

loki.source.file "log_scrape" {
  targets      = local.file_match.local_files.targets
  forward_to   = [loki.write.grafana_loki.receiver]
  tail_from_end = true
}

loki.write "grafana_loki" {
  endpoint {
    url = "http://localhost:3100/loki/api/v1/push"
  }
}
```

Caution: don't use labels too dynamically!

<https://grafana.com/blog/2020/04/21/how-labels-in-loki-can-make-log-queries-faster-and-easier/>

LogQL



QAWARE

Loki has its own query language.

```
count_over_time({foo="bar"}[1m]) > 10
```

```
{host="$host", job="systemd-journal"}  
  | json  
  | line_format "{{ .unit }}: {{ .MESSAGE }}"  
  |= "$search"
```



QA|WARE

Metrics



Metrics: Basics



QAWARE

Metrics are measurements that reflect the current state of the system.

Examples:

- **CPU** load
- Size of the **JVM Heap**
- Number of **calls to an interface**

The metrics are provided by the system to be monitored.

Metrics can also have metadata.

Metrics: Basic - Types of Metrics



QA|WARE

Metrics have different types to visualize data:

- **Counter**

A number that can either be incremented or reset.

- **Gauge**

A metric that can change in either direction.

- **Histogram**

Values in “buckets” - e.g. the duration of requests.

- **Summary**

Similar to histogram, summarizes values and calculates quantiles.

Our metrics storage: Grafana Mimir



QA|WARE

- Mimir (2022) stores **metrics**
- It contains a lot of code from the older tool Prometheus (2012)
- Technical foundation:
 - Time series database for numerical time series
 - Collectors for metrics and text-based format for metrics
 - Query language (PromQL) for time series
 - Alerting based on time series

Metrics: Basics



QAWARE

Grafana Alloy pulls metrics from all configured targets every **15 seconds**.

The transport is usually done via HTTP:

GET /metrics

No aggregation is done here yet. The consumer is responsible for that.

```
# HELP process_open_fds Number of open file descriptors.
# TYPE process_open_fds gauge
process_open_fds 8
# HELP process_start_time_seconds Start time of the process since unix epoch in seconds.
# TYPE process_start_time_seconds gauge
process_start_time_seconds 1.65566242485e+09
# HELP process_virtual_memory_bytes Virtual memory size in bytes.
# TYPE process_virtual_memory_bytes gauge
process_virtual_memory_bytes 1.782685696e+09

# HELP http_server_requests_seconds
# TYPE http_server_requests_seconds summary
http_server_requests_seconds_count{method="GET",status="200",uri="/tle",} 1.0
http_server_requests_seconds_sum{method="GET",status="200",uri="/tle",} 8.183356641
# HELP http_server_requests_seconds_max
# TYPE http_server_requests_seconds_max gauge
http_server_requests_seconds_max{method="GET",status="200",uri="/tle",} 8.183356641
```

PromQL



QAWARE

PromQL is a powerful query language for filtering/aggregating the desired metrics.

The entire time series:

```
http_requests_total
```

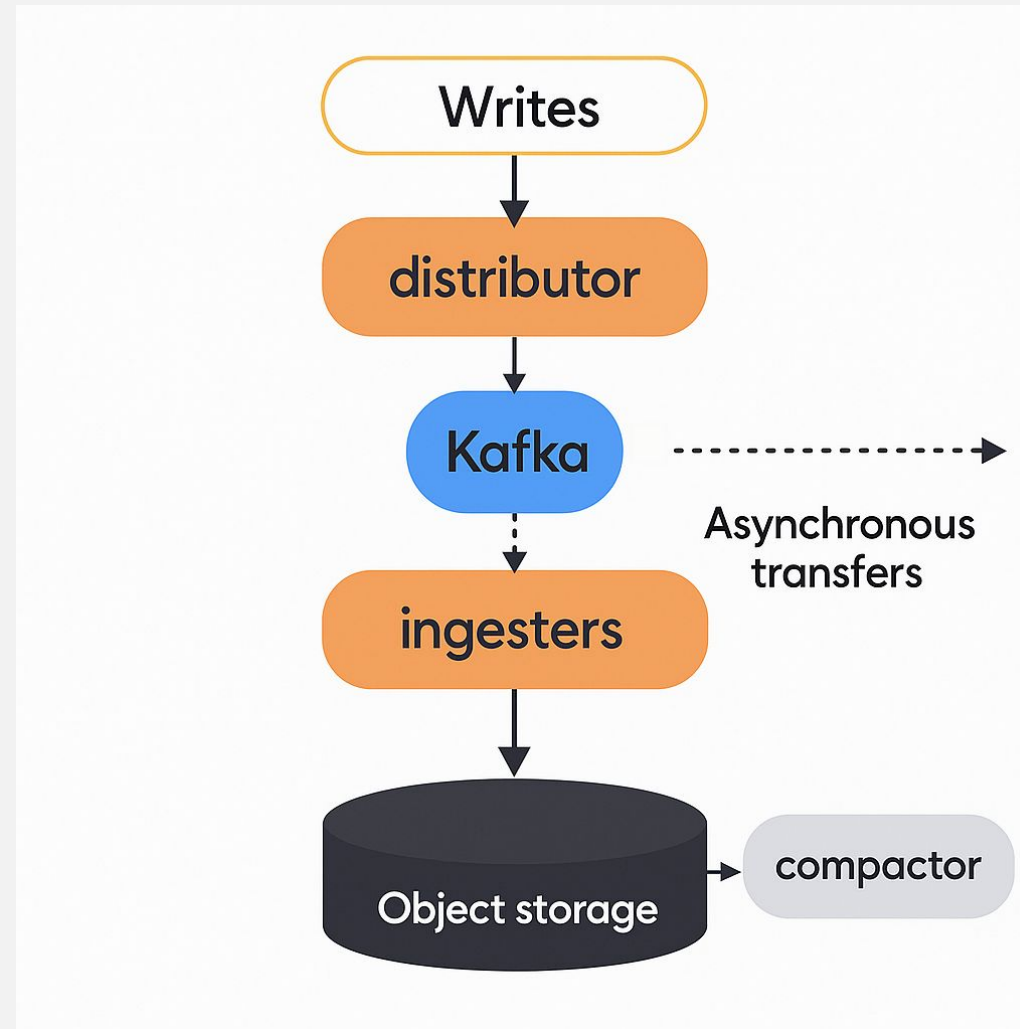
Filter by label:

```
http_requests_total{job="apiserver", handler="/api/comments"}
```

Calculate rate:

```
rate(http_requests_total[5m])[30m:1m]
```

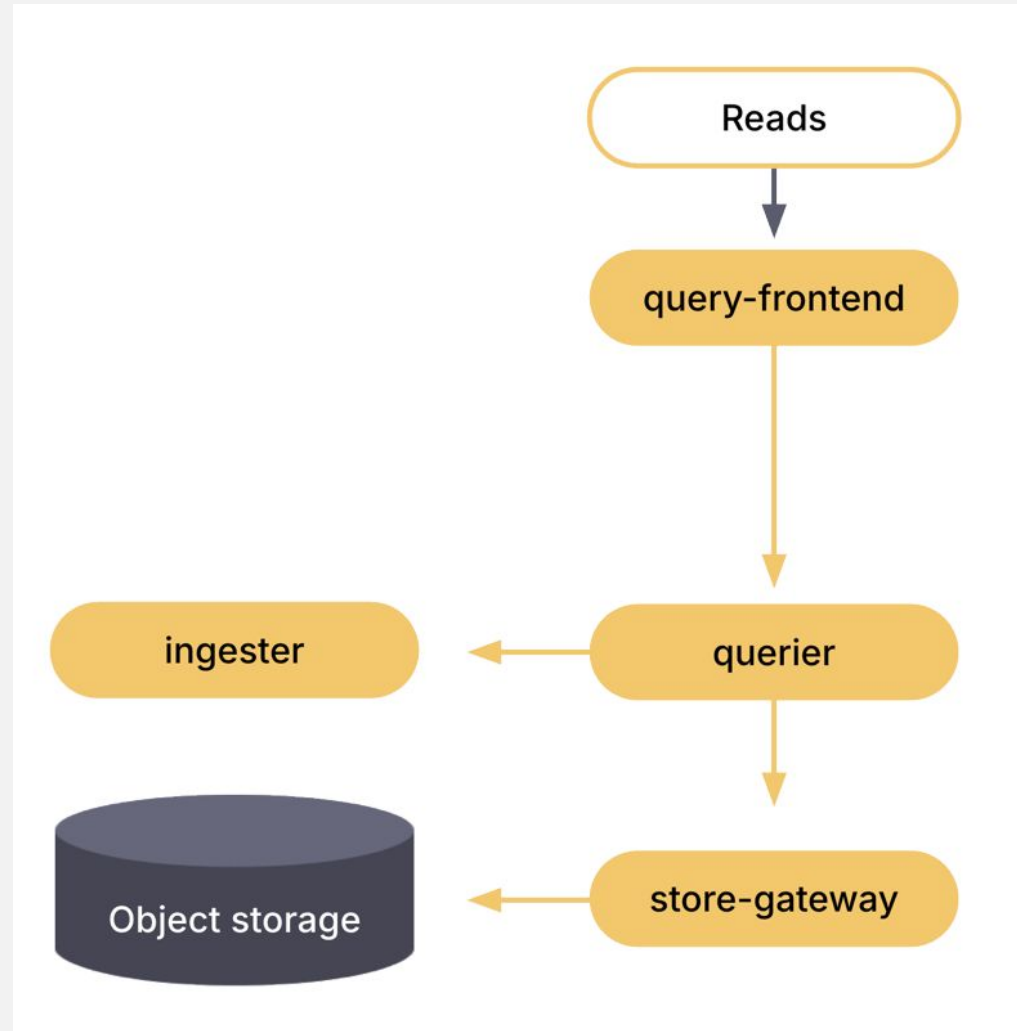
Mimir: Architecture (Write)



Mimir: Architecture (Read)



QA|WARE



Configuration (application)

Example: Quarkus & Micrometer



QAWARE

Gradle:

```
implementation("io.quarkus:quarkus-micrometer-registry-prometheus")
```

Maven:

```
<dependency>  
  <groupId>io.quarkus</groupId>  
  <artifactId>quarkus-micrometer-registry-prometheus</artifactId>  
</dependency>
```


Grafana Alloy



QAWARE

- Scrapes metrics...
 - ...and forwards them to Mimir.
-
- Easy to use (binary + config file)
 - Good integration into Kubernetes, e.g. using the service discovery
 - Configuration in flow mode

Configuration in flow mode (1)



QAWARE

```
// Discover Kubernetes pods to collect metrics from.
discovery.kubernetes "pods" {
    role = "pod"
}

// Scrape metrics from Kubernetes pods and send to a prometheus.remote_write component.
prometheus.scrape "default" {
    targets = discovery.kubernetes.pods.targets
    forward_to = [prometheus.remote_write.default.receiver]
}
```

Configuration in flow mode (2)



QAWARE

```
// Get an API key from disk.
local.file "apikey" {
    filename = "/var/data/my-api-key.txt"
    is_secret = true
}

// Collect and send metrics to a Prometheus remote_write endpoint.
prometheus.remote_write "default" {
    endpoint {
        url = "http://localhost:9009/api/prom/push"

        basic_auth {
            username = "MY_USERNAME"
            password = local.file.apikey.content
        }
    }
}
```



QA|WARE

Traces



Grafana Tempo

Distributed Tracing



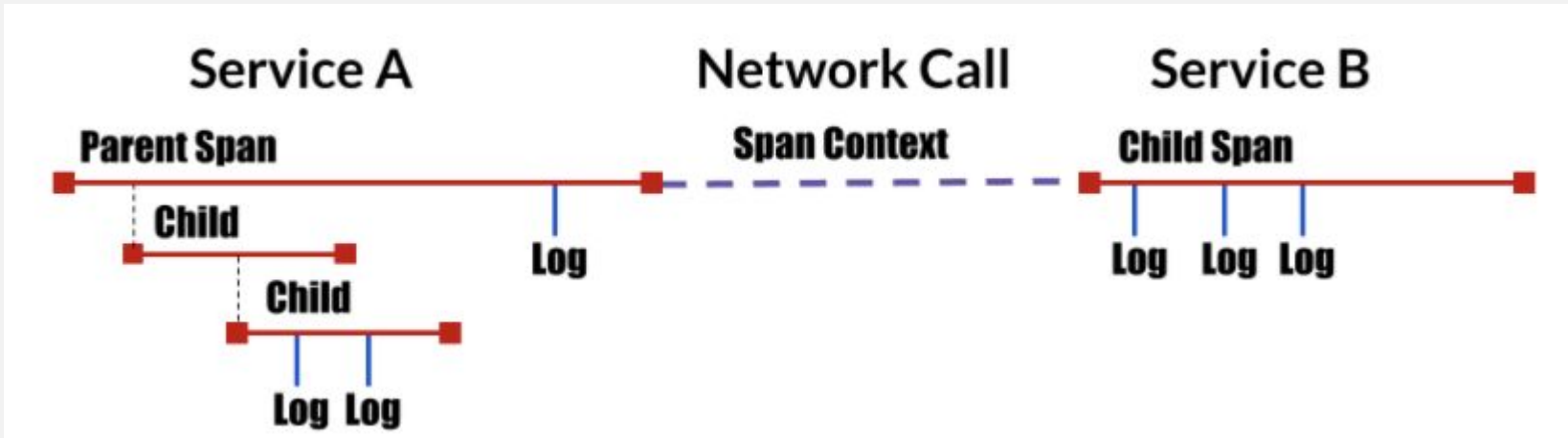
QAWARE

- Distributed Tracing: Technology for tracing calls and processes in distributed software systems.
- Today's considerations go back to the paper: Dapper, a Large-Scale Distributed Systems Tracing Infrastructure
- Idea: Each service forwards information from the caller to the next service and back again. This results in a directed graph.
 - Each service must be instrumented. Otherwise there will be a gap.
 - The smallest unit is called a span. A span has a duration and descriptive information. A trace is a set of 1..n spans.
- Many implementations exist for different programming languages and technologies.
 - <https://opentelemetry.io/docs/languages/>
 - <https://opentelemetry.io/docs/zero-code/>
- By now most users rely on **OpenTelemetry**

Distributed Tracing



QAWARE



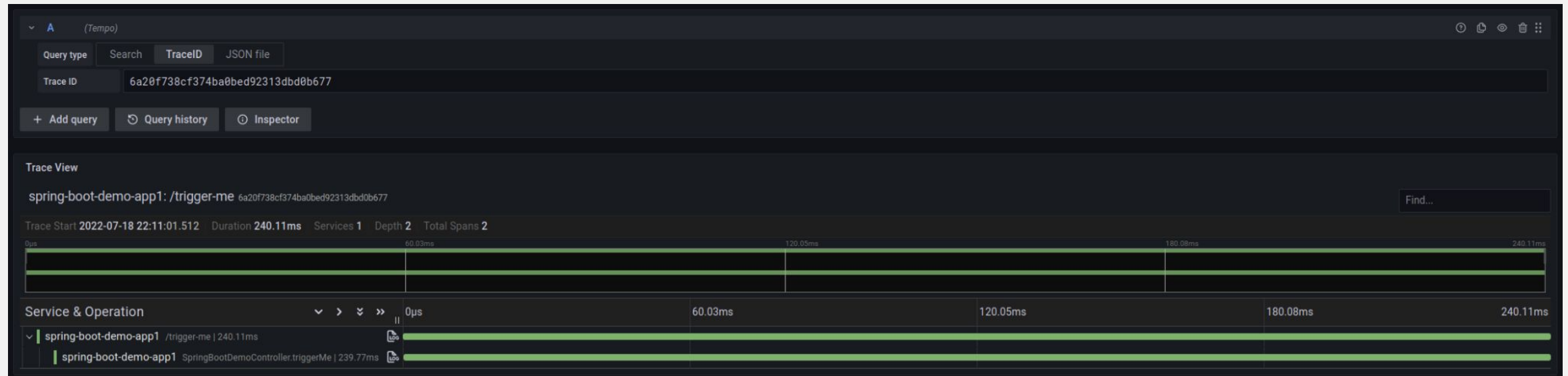
Traces: Foundation



QAWARE

Traces visualize which paths a request has taken through the microservices.....

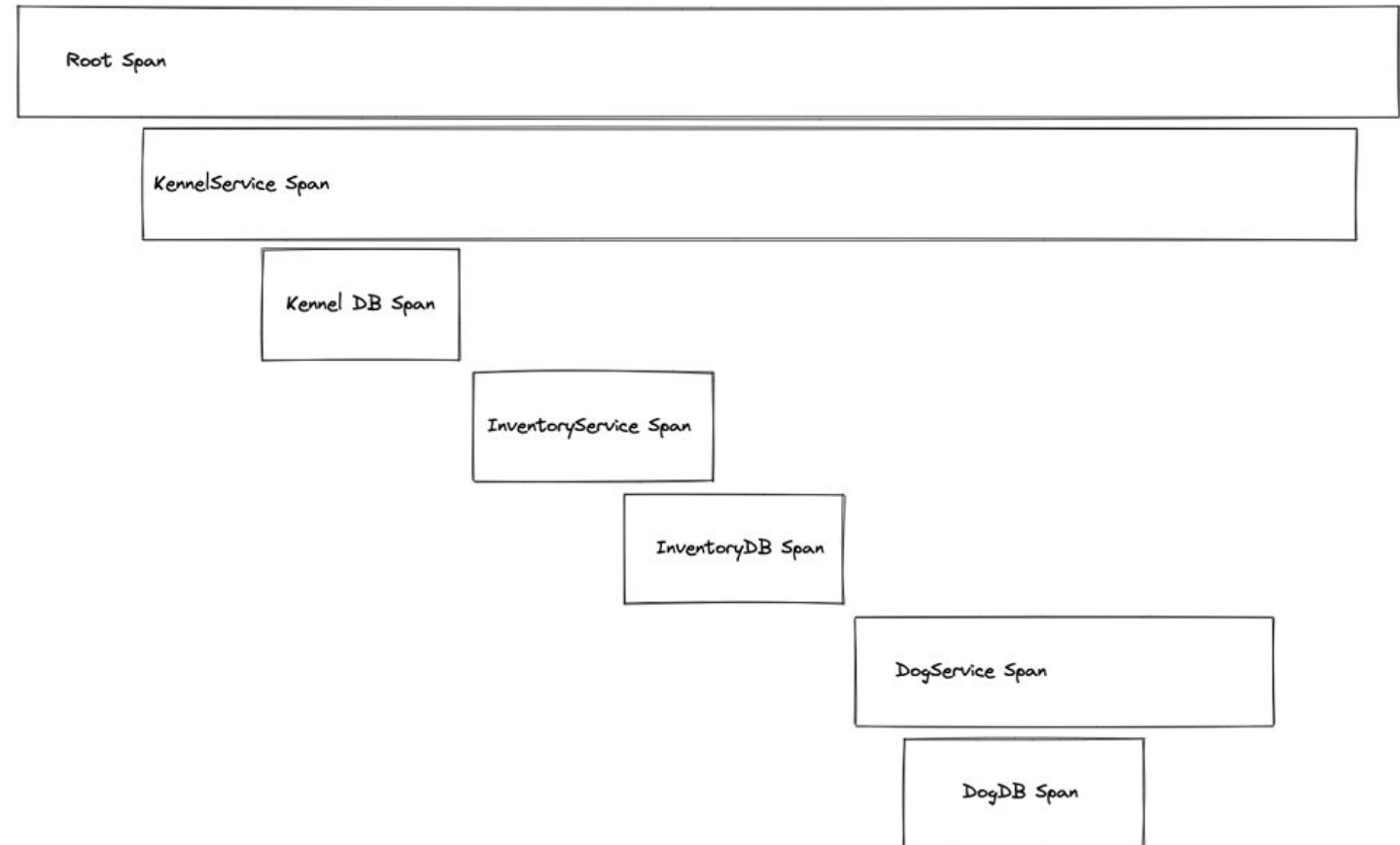
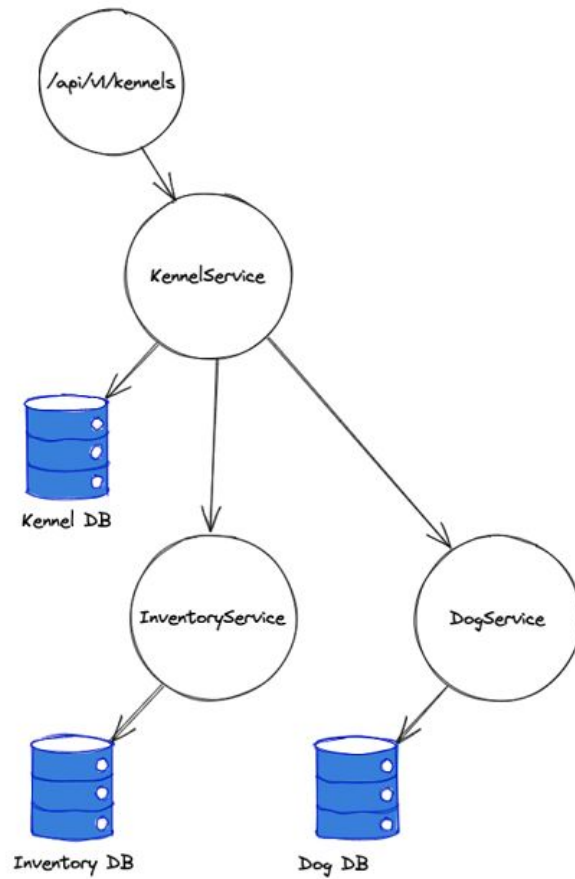
... and where an error may have occurred!



Traces: Grundlagen



QAWARE



Grafana Tempo



QAWARE

Traces in the Grafana ecosystem!

SDKs for various platforms

Compatible with various tracing agents:

- Jaeger
- Zipkin
- Opentelemetry

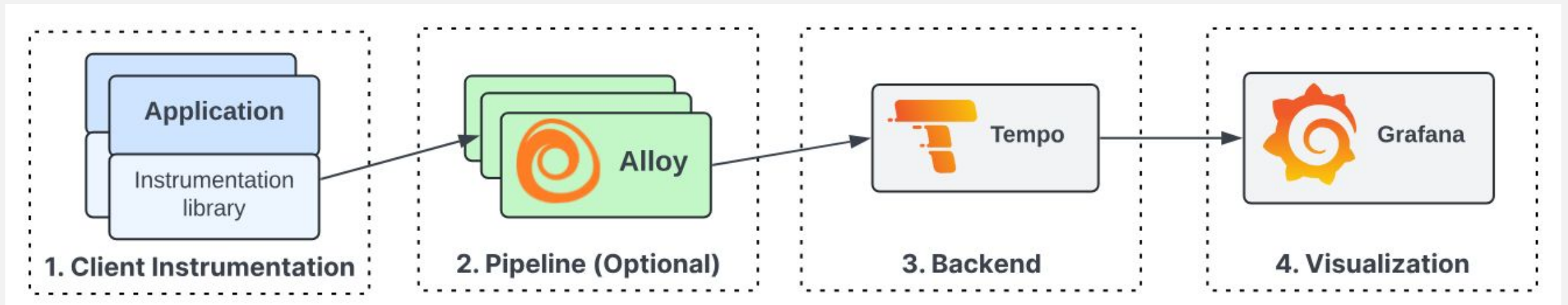
Since: 2020

License: AGPL 3.0

Architecture



QAWARE





QA|WARE

Profiles



Profiling: Basics



Q|WARE

- Measure and analyze the runtime behavior of a program
- Identify resource consumption (e.g. CPU time, memory, or I/O operations) of an application
- Gather information to optimize the program (run faster or with less resources)

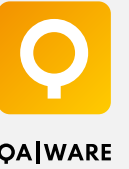
Traditional Profiling:

- Profiler interrupts the program to capture the program's state
- Analyzing these snapshots, developers can deduce the frequency at which parts of the code execute
- Additional code is inserted and may alter the program's behavior due to the added code overhead

Continuous Profiling:

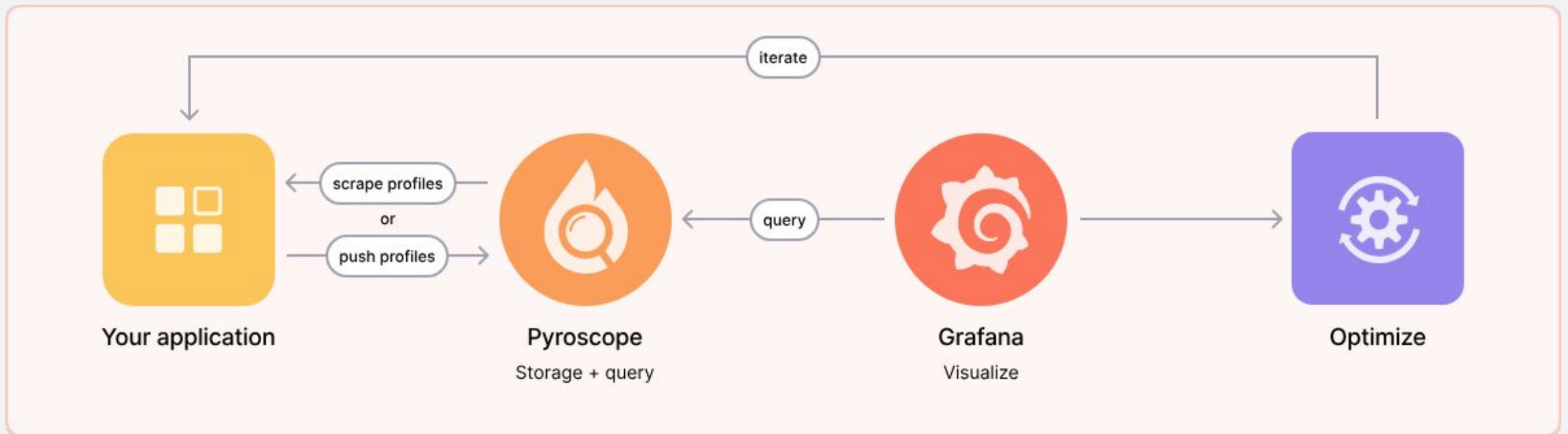
- Profiling data is continuously collected in the background with minimal overhead
- Developers gain a more comprehensive view of a program's behavior over time
- May be less detailed than traditional profiling due to the minimal intrusiveness

Grafana Pyroscope



Profiles in the Grafana ecosystem!

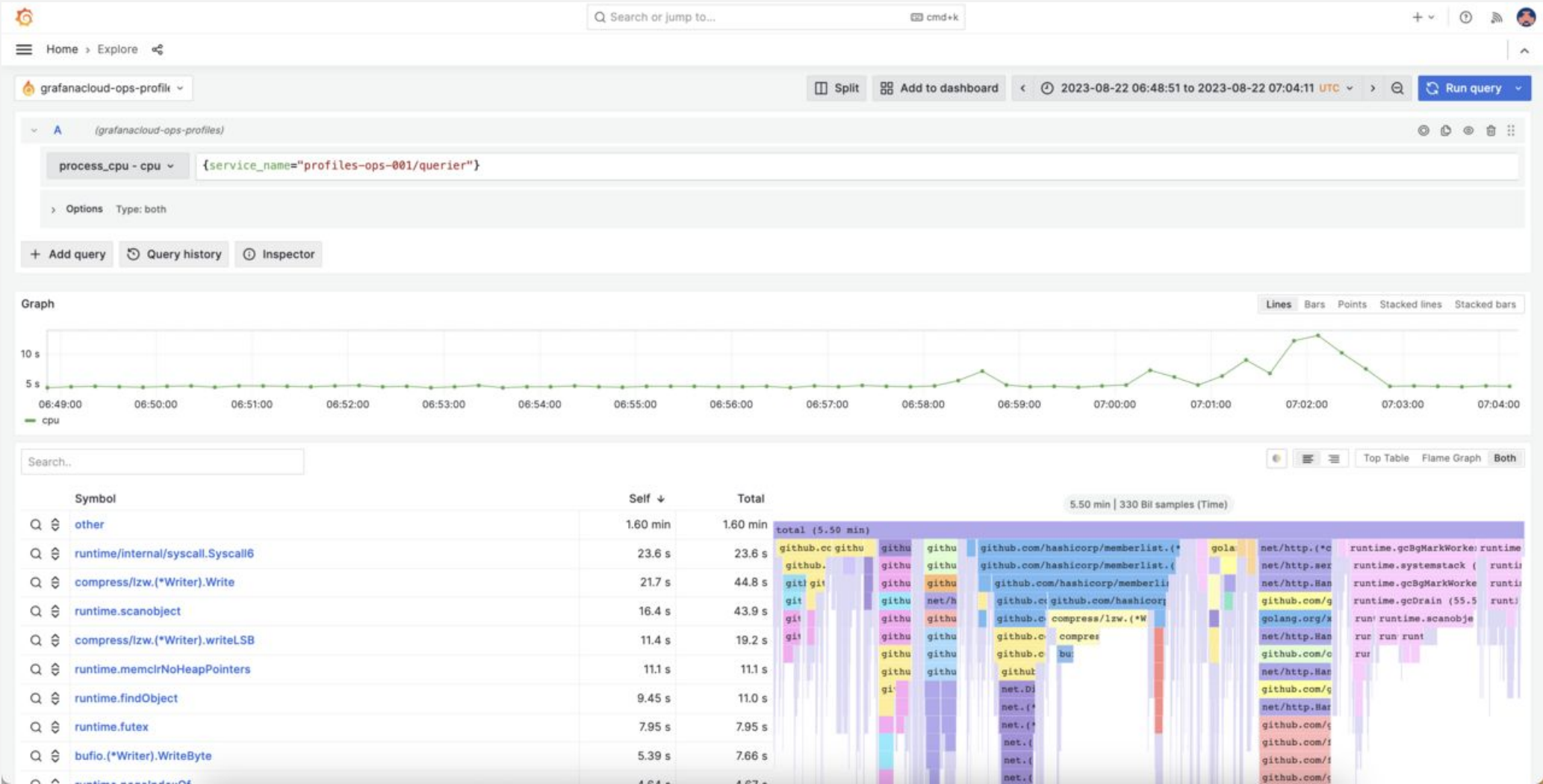
Since: 2022



Grafana Pyroscope



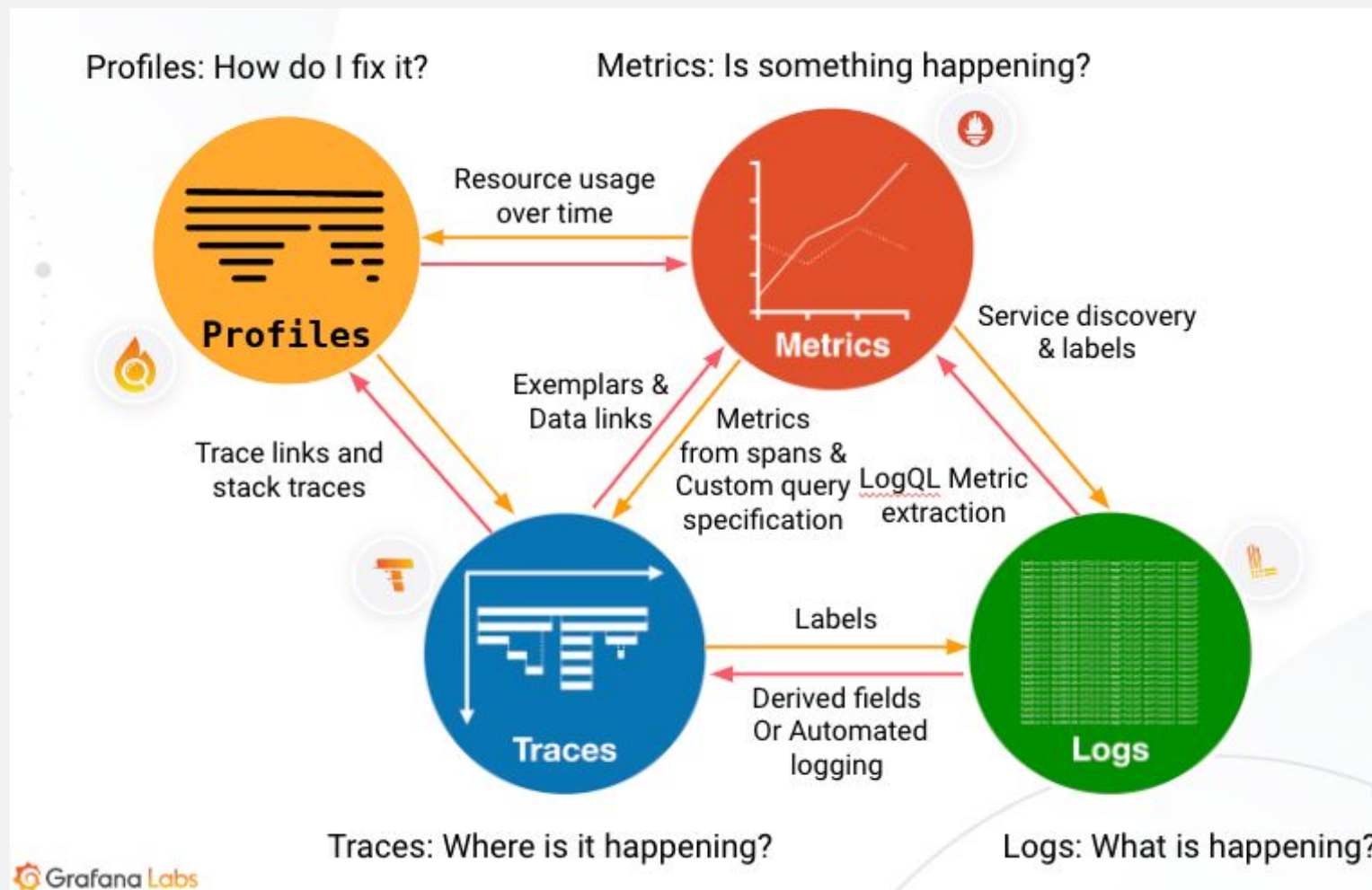
QAWARE



Combining different data source in Grafana



QAWARE





QA|WARE

Visualization



Grafana



QAWARE

Visualization for many different data sources:

- Mimir
- Loki
- Tempo
- Pyroscope
- InfluxDB
- ...

Since: 2014

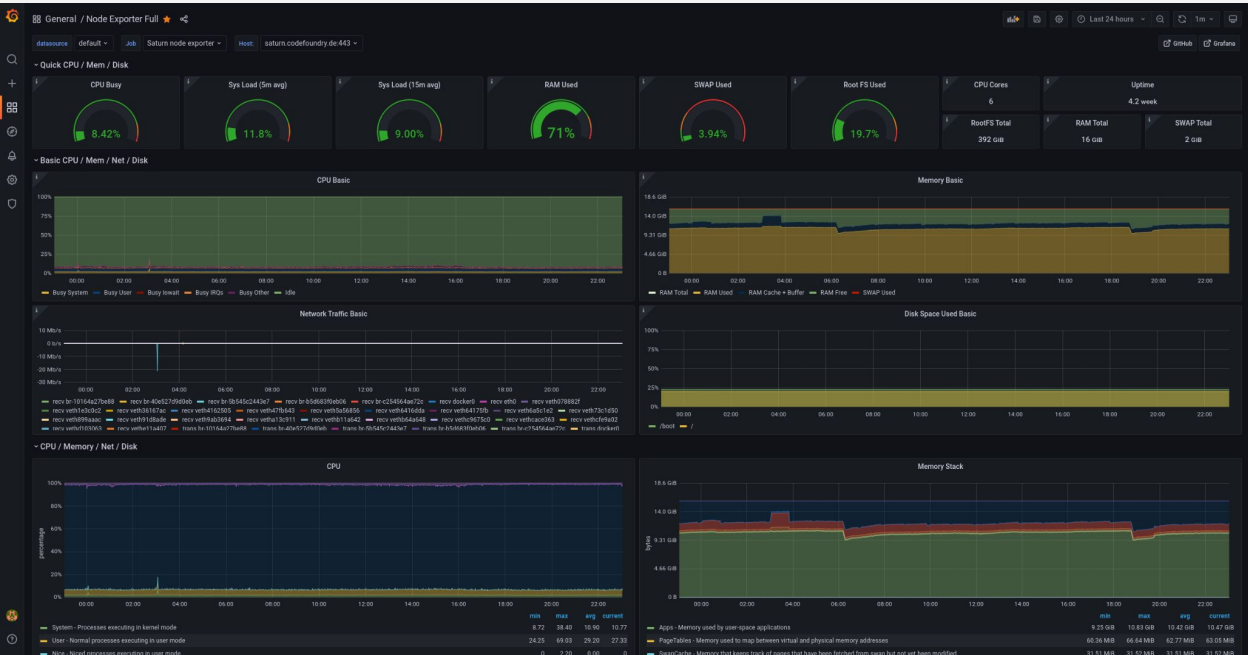
License: AGPL 3.0

Dashboards



QAWARE

Data of all types is visualized in dashboards.



Grafana - Dashboards



QAWARE

Don't worry: you don't have to laboriously click together dashboards yourself.

<https://grafana.com/grafana/dashboards/>



QAWARE

More from the Grafana universe

Load tests: k6



QAWARE

Built with Go – the tests are written in JavaScript.

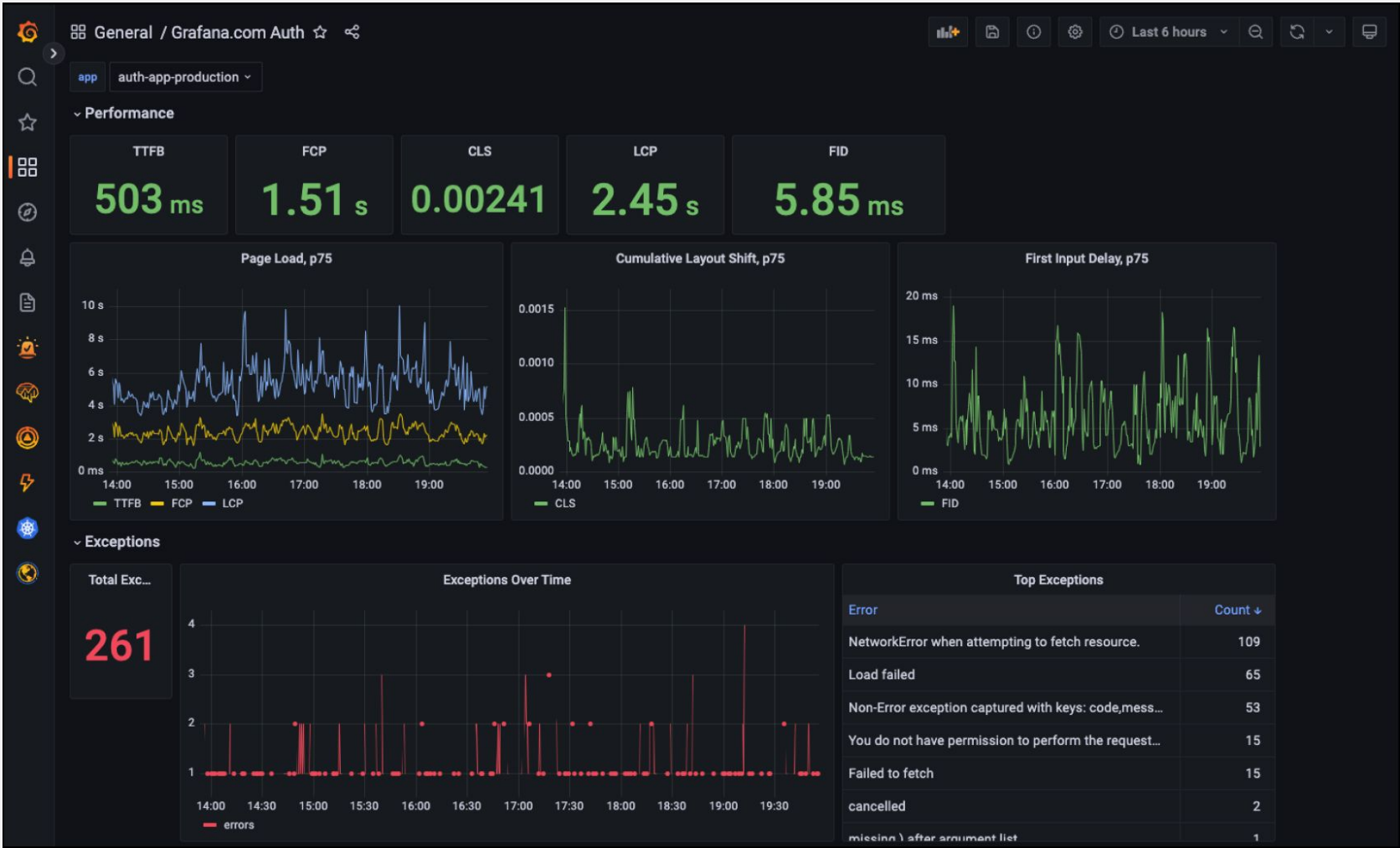
Test results end up in an InfluxDB – visualization is done with Grafana.



Frontend observability: Faro



QAWARE



<https://grafana.com/oss/faro/>

Further reading



QA|WARE

Blog: <https://blog.qaware.de/posts/cloud-observability-grafana-spring-boot/>

Showcase: <https://github.com/zalintyre/cloud-observability-grafana-spring-boot>

Grafana @ Heise Mastering Kubernetes:

<https://de.slideshare.net/QAware/cloud-observability-mit-loki-prometheus-tempo-und-grafana>

Grafana: <https://grafana.com/>

Mimir: <https://grafana.com/docs/mimir/latest/>

Loki: <https://grafana.com/oss/loki/>

Tempo: <https://grafana.com/oss/tempo/>

Pyroscope: <https://grafana.com/oss/pyroscope/>

Sources



QAWARE

[Kal] R.E. Kalman,
On the general theory of control systems,
IFAC Proceedings Volumes,
Volume 1, Issue 1,
1960,
Pages 491-502,
ISSN 1474-6670,
[https://doi.org/10.1016/S1474-6670\(17\)70094-8](https://doi.org/10.1016/S1474-6670(17)70094-8).
(<https://www.sciencedirect.com/science/article/pii/S1474667017700948>)

qaware.de



QA|WARE
SOFTWARE ENGINEERING

QAware GmbH

Aschauer Straße 32
81549 München
Tel. +49 89 232315-0
info@qaware.de



twitter.com/qaware



linkedin.com/company/qaware-gmbh



xing.com/companies/qawaregmbh



slideshare.net/qaware



github.com/qaware